

# Project Action Plan: Predicting House Prices Using Advanced Regression Techniques

## 1. Project Overview

### Project Title

Predicting House Prices Using Advanced Regression Techniques

### Objectives

- **Primary Objective:** Build a robust, highly accurate machine learning pipeline to predict residential home sales prices based on 79 explanatory features detailing structural, spatial, and qualitative aspects of the properties.
- **Secondary Objective:** Identify and quantify the key drivers of property value (e.g., location, total square footage, material quality) to provide interpretable business insights.

### Scope

- **In-Scope:** Comprehensive data cleaning and missing value imputation, advanced feature engineering (log transformations, interaction terms, target encoding), evaluation of multiple regularized linear and tree-based ensemble models, hyperparameter optimization, and deployment of a local interactive interface.
- **Out-of-Scope:** Setting up real-time production database pipelines or building automated continuous retraining triggers.

### Team Roles

- **Lead Data Scientist:** Responsible for the overall modeling strategy, feature engineering architecture, and final ensemble model tuning.
- **Data Analyst:** Dedicated to initial exploratory data analysis (EDA), statistical validation, correlation mapping, and reporting insights.
- **Machine Learning / DevOps Engineer:** Focuses on productionizing the model, writing clean inference pipelines, and building the web application interface.

## 2. Detailed Timeline (4-Week Plan)

### Week 1: Data Ingestion & Exploratory Data Analysis (EDA)

- **Day 1–2:** Set up repository structures, ingest the Kaggle dataset, profile data types, and map out missing value distributions.

- **Day 3–4:** Analyze target variable (SalePrice) distribution; apply log/Box-Cox transformations to fix skewness. Conduct univariate and bivariate analyses.
- **Day 5:** Construct correlation matrices, evaluate multi-collinearity (Variance Inflation Factor), and finalize the EDA notebook.
- **Milestone 1:** *Comprehensive EDA report completed; data-driven plan for missing value handling established.*

## Week 2: Advanced Feature Engineering & Selection

- **Day 6–7:** Execute missing data imputation strategies (e.g., using median values by neighborhood for missing lot frontage). Handle ordinal and nominal categorical encodings.
- **Day 8–9:** Detect and drop extreme spatial outliers. Construct strategic interaction terms (e.g., combining individual square footage columns into TotalSurvivableSF).
- **Day 10:** Normalize numeric features using robust scaling methods and conduct initial feature importance filtering.
- **Milestone 2:** *Final clean feature matrix generated and exported, ready for model training pipelines.*

## Week 3: Model Building, Tuning & Ensembling

- **Day 11–12:** Establish a rigid Out-of-Fold (OOF) cross-validation framework (5-Fold Stratified CV). Train baseline models (Ridge, Lasso, ElasticNet).
- **Day 13–14:** Train non-linear models including Random Forests, Gradient Boosting, XGBoost, and LightGBM. Leverage Optuna for automated hyperparameter tuning.
- **Day 15:** Build an ensemble architecture combining top-performing linear and tree models via a meta-learner (Stacking) or optimized weighted blending.
- **Milestone 3:** *Optimized final model saved, meeting or exceeding baseline validation benchmarks.*

## Week 4: Deployment, Testing & Documentation

- **Day 16–17:** Develop a lightweight, user-friendly interactive web application using Streamlit to serve live model predictions.
- **Day 18–19:** Refactor code into clean .py production scripts. Write unit tests for data preprocessing pipelines and compile the final research report.
- **Day 20:** Conduct comprehensive code review, finalize documentation within the GitHub repository, and hold a project retrospective.

- **Milestone 4:** *End-to-end delivery of the code repository, interactive application, and evaluation reports.*

### 3. Resource Requirements

#### Software & Libraries

- **Environment:** Python 3.9+ runtime environment
- **Data Manipulation:** NumPy, Pandas
- **Visualization:** Matplotlib, Seaborn
- **Machine Learning:** Scikit-Learn, XGBoost
- **Optimization & Deployment:** Streamlit or Flask
- **Version Control Infrastructure:** Git paired with a secure GitHub repository.

#### Hardware Requirements

- **Processor (CPU):** Minimum Quad-Core Intel i5 / AMD Ryzen 5 or equivalent Apple Silicon configuration.
- **System Memory (RAM):** 8 GB minimum (16 GB recommended to handle grid search cross-validation in parallel threads).
- **Storage Space:** 2 GB of free local disk space for datasets, dependencies, and serialized pipeline binaries.

#### Dataset Source

- **Primary Source:** Kaggle House Prices: Advanced Regression Techniques dataset (train.csv, test.csv, and data\_description.txt).

### 4. Risk Assessment & Mitigation

| Risk Description   | Impact | Probability | Mitigation Strategy                                                                                                                   |
|--------------------|--------|-------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Severe Overfitting | High   | High        | Enforce rigid 5-fold cross-validation, utilize L1/L2 regularization penalties, and set conservative tree depths in gradient boosting. |

| Risk Description                        | Impact | Probability | Mitigation Strategy                                                                                                       |
|-----------------------------------------|--------|-------------|---------------------------------------------------------------------------------------------------------------------------|
| High Data Sparsity / Missing Values     | Medium | High        | Apply context-specific imputation (e.g., replacing missing pool quality flags with "No Pool" rather than the mode value). |
| Data Leakage During Feature Engineering | High   | Medium      | Build all preprocessing transformers explicitly inside Scikit-Learn Pipelines to fit exclusively on training folds.       |
| Deployment Latency / Model Size         | Low    | Low         | Keep ensembling structures lightweight (blend 3–4 models max) and serialize artifacts using optimized compressions.       |

## 5. Deliverables Checklist

- ☐ **Research Report:** A comprehensive summary detailing structural findings, modeling methodologies, and key business insights.
- ☐ **Exploratory Data Analysis (EDA) Notebook:** Well-documented Jupyter notebook outlining data cleaning logic, visual distributions, and statistical findings.
- ☐ **Model Building & Tuning Notebook:** Code showing cross-validation runs, hyperparameter sweeps, and model performance comparisons.
- ☐ **Interactive Web Application:** Production-ready Streamlit application file allowing manual user input for real-time price evaluation.
- ☐ **Production GitHub Repository:** Structured, clean repo featuring a functional README.md, requirements.txt, configuration modules, and unit tests.

## 6. Success Metrics

1. Predictive Precision: Target Cross-Validated  $R^2$  Score  $> 0.85$
2. Error Minimization: Log-Transformed RMSE  $< 0.12$  on Test Split
3. Processing Overhead: Inference response latency under 200ms

## Model Comparison Criteria

Models will be thoroughly benchmarked across the following criteria to choose the absolute best candidate for the stacking pool:

- **Generalizability:** Low delta between training error and validation error metrics.
- **Computational Efficiency:** Training time vs. accuracy tradeoff, particularly during extensive grid searches.
- **Feature Interpretability:** Accessibility of feature importance coefficients or SHAP values to explain predictions easily to non-technical stakeholders.